

Alt Theory 用户文档（简体中文）

本目录与英文 Helper 文档平行，供熟人阅读与 PDF。应用内 Helper 只读英文 docs。

生成：node scripts/docs-zh-build.mjs PDF：npm run docs:zh-pdf（需要 Pandoc）

页数：38（与英文 corpus 对齐）。

Alt Theory 文档（简体中文）

文档来源与状态

本套文档由 AI agent 根据产品代码、开发过程文档与笔记生成，再经人工校对。人工校对仍在进行；(planned) 功能与产品本身仍在演进，**内容可能有误差**。若与实际应用或仓库代码不一致，以应用与代码为准。

英文对照见上级目录 [en/](#)。

可调用名（技能 id、slash 命令）以英文 id 为准，中文仅作括号释义。目前没有中文别名。

从这里开始

- [简介](#)
- [安装与启动](#)
- [开始第一次对话](#)
- [理解模式与工作模式](#)
- [带入材料与上下文](#)
- [继续、转向与稍后返回](#)
- [应用与外挂](#)

使用 Alt Theory 应用

- 以理解为目标
- 在关键时刻想得更清楚
- 以具体工作为主任务
- 应对不确定性

系统指南

- 对话与历史
- 模型、提供方与访问
- 工作目录、文件与路径
- 知识库与上下文
- 处理回复与控制
- 检索、来源与网页内容
- 文档、图像与其他输入
- 技能如何工作
- 内置技能
- 添加与管理技能
- 命令
- 工具箱
- 权限、授权与智能体活动
- 智能体团队与子代理会话
- 导入、交接与跨工具连续性
- 助手与界面内引导
- 设置与配置

进阶

- 外挂与能力差异
- 跨工具工作与连续性
- 共享配置、技能、模型与资产
- 不修改 Alt Theory 的自定义
- 修改 Alt Theory 及其扩展边界
- 兼容性、更新与集成调试

帮助与参考

- [常见问题](#)
- [兼容性、格式与已知限制](#)
- [术语](#)

会话级来源说明见 [README-SOURCE-SESSION.md](#)。

简介

Alt Theory 是社会科学研究者在 AI 快速发展时代主导思考「AI 应具备何种品质」而形成的设计尝试，其设计功能与哲学随 AI 的发展而演化。

- 2026.05. Alt Theory 起步时是环境心理学领域的理论知识库，加上一个用于理论创新的 thinking partner。它回应了当时 ai agent 与手动编排工作流的能力，致力于提高领域经典知识获取的可靠性。
- 2026.12. Alt Theory 的第一个版本在 dify 中实现，包含环境心理学领域基于 RAG 的理论知识库与元理论思维的引导。
- 2026.04-2026.06. 随 coding agents 能力增强，Alt Theory 纳入 coding agent 能力，保留 understanding-first 的哲学。其行为方式扩展为：对来源和不确定诚实、保持合适推进节奏 (pacing) 等。并加入读取用户文件、修改文件、总结等结合用户工作进行讨论的能力。在”建立理解”作为推进有价值工作的关键时刻发挥作用。
- 2026.07. 额外增加了“工作模式”以支持与“理解阶段”交错的“工作阶段”。可导入其他 agent 工具的对话，进行连续性的数据分析、转换读取文献等文件，可通过讨论帮助用户逐步推进计划、定向、action for reflection。可让文档记录与文件夹管理更可溯源、可复现，以支持社会科学与广义研究活动。

与通用chatbot的差异

Alt Theory 的行为分三层：身份层 (soul) 规定原则与世界观，维持不变；角色 (role) 规定呈现方式与发言风格，可以切换和扩展；技能 (skill) 规定在受情境条件约束的特定任务上的处理方式，可以自动触发和便捷手动触发。例如，adaptive-aligning (自适应对齐) 在对情况、目标、和方向未达成共同理解时，与用户对齐。

由此，它可以把沟通、文档与研究中的不确定问题当作要主动应对的目标，而不是要掩盖的弱点。

默认行为

Alt Theory 的身份层和默认角色规定以下原则，由智能体在所有对话中持续遵守：

- 原则优先于顺从。智能体对用户成功判断是基于「在上下文中是否守住了原则」，而非「当下是否让用户满意」。
- 事实与不确定性优先于附和。出现事实、时间或归属错误时，智能体会直接指出，并标注已确立、合理但未核实、或未知。
- 暴露逻辑和信息缺口，不抹平。用户的 framing 与证据不完全契合时，智能体会摆出差距（哪里已知、哪里推断、哪里未定），不为了合理理论而修剪边缘。
- 不过度拟合或归因。智能体不把” 可以看作” 提升为” 是” 的论点。
- 识别讨好倾向，不服从它。智能体识别「给个答案、附和、给出结论」这些生成倾向是拉力而非命令，按原则自主选择。
- 为未解决留空间。智能体不急于解决用户的疑问或矛盾；对话中的不适不是靠让步来修复的问题。
- 默认对陈述标明来源：现场查得、记忆提取（未核实）、或推断所得。记忆内容不会以检索结果的形式呈现。

触发行为

Alt Theory 的技能（skills）均可以自动触发，也可以有用户按需简易手动触发（planned）。不同技能在不同的角色设定中有不同侧重，并持续扩展。

规划、对齐和问题理解技能（英文 id 为可调用名，如 /adaptive-aligning）：

- adaptive-aligning（自适应对齐）：在定方向工作之前，让智能体先以成批问题（每个带自己的最佳猜测）与用户对齐，使用户原本的设想与它的理解之间的偏差暴露出来。
- adaptive-plan-record（计划记录）：为多步工作建立活计划，跨会话保持。
- workspace-conventions（工作区约定）：决定智能体在项目内创建文件的位置与命名，并且根据问题的确定和不确定提出灵活可扩展的文件系统结构。
- conversation-summary（对话摘要）：用户要求摘要、交接说明、或提取长对话关键信息时调用，仍包含信息来源的标注。
- search-policy（检索策略）：规范实时查核的时机与来源标注；理解模式下管辖无查核时的回退。

基础工具技能：

- web-search (网页检索, 工作模式)：实时通用与学术检索。
- page-fetch (页面抓取, 工作模式)：把 URL 抓取为可读文本，按 search-policy 引用。
- doc-convert (文档转换, 工作模式)：文档格式转换与生成。
- precise-edit (精确编辑)：处理接近定稿文本时（用户发送已成形文稿做语言润色）启用克制编辑。

APP 设置和使用帮助：

- imported-session-context (导入会话上下文)：在导入自其他工具的会话中启用，处理来源缺失与恢复。
- alt-theory-help (应用帮助)：关于 Alt Theory、环境设置、bug 与模型能力检查的唯一帮助入口。

理解模式与工作模式

- 理解模式是为传统 ai 应用用户准备的以讨论和文档记录讨论为主导的模式，通过有限的权限提高安全性。
- 工作模式在同样的”应对、解决不确定”的基础之上，增加了实时检索、文档生成与指令执行，支持与“理解阶段”交错的“工作阶段”。

例如，其可以连接本机已经配备的程序和工具，讨论研究假设和问题（理解），进行探索性的r、python数据分析（工作），再讨论分析结果的意义、对研究问题的回应（理解），制作后续的ppt和表格和与合作者讨论的材料（工作）等。详见[理解模式与工作模式](#)。

适用用户

适用于社会科学领域的学生与研究者，从第一次构思问题的硕士生，到判断某工具是否该写进研究计划的资深研究者。从理论和研究设计讨论、到完成实际任务，都可以在不同模式得到支持。详见[理解模式与工作模式](#)。

同样适用于泛研究类知识工作，可以通过设置为更加接近一般知识工作要和编码工具的模式。(planned)

使用本应用不要求任何ai代理工具经验。若 Alt Theory 的用户已在用这类工具，可以通过内建的导入功能，简单导入已有对话。支持的harness：

- codex

- [claude code](#)
- [grok build](#)
- [pi coding agent](#)
- [opencode](#)

详见[应用与外挂](#)。

延伸阅读

- [安装并启动 Alt Theory](#)
 - [使用 Alt Theory 应用](#)
-

安装与启动

Alt Theory 在本机运行，获取方式有两种。

打包应用

在内部测试阶段，打包后的桌面应用直接发给测试者与协作者（现阶段），公开测试阶段在 [github release](#) 可获取。

支持以下平台：

- 配备 Apple Silicon 的 macOS
- x86 架构的 Windows

其他组合未经测试，不作保证。

首次启动时，操作系统可能警告该应用来自未识别的开发者，因为它未经平台应用商店分发。在 macOS 上，首次打开请右键点击应用并选择「打开」。在 Windows 上，于 SmartScreen 对话框中选择「更多信息，然后仍要运行」。下面从源码构建可避开此提示。

从仓库构建（需一定编程经验）

完整源码公开，使用当前版本的 Node.js 与 npm。

```
git clone https://github.com/syuan-research/alt-theory
cd alt-theory
npm ci
npm --prefix alt-theory-app/frontend ci
npm run build:frontend-v6
```

用平台启动脚本启动本机应用（当前命令见仓库 README）。应用提供一个本地 Web 界面并在窗口中打开，不托管在任何服务器上。

首次启动

第一屏就是应用本身，一段等待开始的对话。应用至少需要配置一个模型提供方，之后才能开始对话。Alt Theory 提供工作目录与方法，模型由用户配置的提供方提供。

- 若应用来自 Alt Theory 团队，提供方可能已配置好。
- 否则首次启动的设置界面会引导用户添加提供方，例如一个 API 密钥，或用受支持的订阅登录。详见[模型、提供方与访问](#)。应用内的[助手](#)可逐步引导。

费用

本软件免费。模型按用量、按提供方定价收费。推荐默认是 MiMo 2.5 Pro。它不是最新模型，但贴合本产品的行为要求，且性价比高。MiMo 2.5 Pro 搭配 OpenCode Go 订阅，中高强度使用每月约 10 美元。直接使用 MiMo API 轻度使用每月不到 5 美元。Codex 订阅也可用。GPT 系列模型在反思性研究工作中比 MiMo 2.5 Pro 更被动。价格会变，请查阅模型页获取当前数字。

数据去向

- 对话保留在本机。
- 用户配置的模型会收到对话文本，方式与任何 AI 模型相同。这正是它能回复的原因。
- 除非用户选择导出，否则没有其他内容离开本机。

对话数据是单一应用数据目录下的普通本地文件。复制该目录即可备份全部内容，把它放到另一台机器上，应用会自动识别。

提供方激活后，可进入[开始第一次对话](#)。

首次对话

新对话从问候、两个模式卡片（理解与工作）和消息输入框开始。选择理解模式（细致对话，不会触及任何文件），开始输入。

第一条提示

Alt Theory 在真实、尚未成形的思考上才显出本色，以下提示为例。

我好奇为什么人们在公园改造后还以不同方式使用它，但我说不清自己的问题是关于场所依恋、习惯，还是别的什么。你能帮我弄清自己到底在问什么吗？

通用聊天机器人会直接丢给用户一个框架。此处首次回复更可能放慢，区分概念、询问用户观察到的内容、标注不确定的地方。

回复内容

思考、工具行（读取、检索、技能使用），以及就地渲染的表格与图示。详见[处理回复与对话控制](#)。

下一步

模式卡片是应用中最重要选择，详见[理解模式与工作模式](#)。

理解模式与工作模式

每段对话都在两种模式之一中运行。两者的区别在于本应用向智能体开放了多少本机与工具生态。两种模式都具有代理能力。

理解模式

细致对话，触及范围受限。

- 仅与对话、所附文件、所选知识库，以及它自己的知识（已标注）协作。

- 不浏览网页，不运行命令，也不漫游文件系统。
- 无实时查核。某条陈述需要核实时，智能体会如实说明，并按可信程度标注其当前最佳理解（已确立、合理但未核实、或未知）。记忆内容不会以检索结果的形式呈现。

理解模式是使用 Alt Theory 的一种完整方式，而非工作模式的预览。

工作模式

保持同样的对话标准，并增加以下实际触及范围。

- 一个工作目录（可不止一个），智能体可在其中读取与创建项目文件
- 实时检索与抓取（通用网页、带可核元数据的学术检索，以及整页获取）
- 文档工作（读取 Word、PDF、Excel、PowerPoint，并生成新文档）
- 命令与安装，均经过可见的授权流程

工作模式将同样的标准带入行动。重要选择会加以说明，来源得以保留，原始文件始终保持原样，不会被就地改动。

需要授权的操作

- 在附上的工作目录内，智能体逐步工作，不就每一步打断用户。
- 读取这些目录之外的内容、超出内置检索工具的联网访问，以及任何安装，都会弹出授权请求，说明请求的内容与所授予的权限。

完整说明见[权限、授权与智能体活动](#)。

切换模式

可随时在理解模式与工作模式间切换一段对话。切换从下一条消息生效。对话、推理、所附材料、角色与知识库全部随之转移。切换从不派生分支，也不重置任何内容。

模式选择

希望仅保留思考、不让其他东西移动时，选理解模式。希望对话能触及文件与实时世界时，选工作模式。答案改变时随时切换。

导入材料与上下文

把材料接入对话，最低限度需要了解以下概念。

组成部分

- 对话是用户与智能体思考和工作之处。它保留自身的历史与配置。
- 文件附件把某个具体文件放到智能体面前，仅供本场对话使用。拖入输入框或使用附上按钮。
- 工作目录（工作模式）是一个本机文件夹，智能体可在其中工作，即读取文件并新建文件。附上一个文件夹，就定义了实际工作发生的地方。
- 知识库是精选的参考资料。相关领域适用时启用它。

这些可以组合，例如一场对话可能附上一份草稿、以一个项目文件夹作为工作目录，并启用一个知识库，也可以都不用。

智能体可见内容

智能体看到所接入的内容，包括附件、工作目录的内容、启用的知识库，以及写下的文字。它看不到本机的其余部分。输入框区域显示本场对话的工作目录与知识库。

真实文件

Alt Theory 读取科研常用的格式（Word、PDF、Excel、PowerPoint），转换后的副本以 `_converted` 后缀出现在原件旁边。图片以同样方式附上，智能体能否看见它们取决于所选用的模型。详见[文档、图像与其他输入](#)。

智能体修订文档时遵循一条约定，写入一份副本或一个命名清晰的新文件，并告知写的是哪一个。原文件保持原样。

完整说明在哪里

- 文件夹与路径：[工作目录、文件与路径](#)
 - 知识库：[知识库与上下文](#)
 - 格式与图像：[文档、图像与其他输入](#)
-

继续、转向与稍后返回

研究很少一路直线走完。它会暂停、分叉、逆转，数周后再拾起。Alt Theory 的对话正是为应对这种节奏而设计。

对话持久保存

每场对话都保存在本机，出现在对话列表中，按工作目录分组且可检索。对话在首次交换后自行命名，随时可重命名。重开任意对话并继续，其上下文、材料、模式与配置随之恢复。

转向

这些控制都是非破坏性的，每一项都在原件旁开启一个版本，原件从不修改，便于比较。

- 停止（随时中断一条回复）。
- 编辑（改写早先的一条消息，以新措辞从该点重跑讨论，观察不同的切入点是否改变答案）。
- 分叉（开启一场相关对话，基于同一份累积上下文追求不同方向）。
- 顺带（BTW，开启一个侧边对话，携带当前上下文，用于不该污染主线的旁支）。

这些操作开启的比较面板见[处理回复与对话控制](#)。

长对话仍能工作

长对话最终会超出模型一次能容纳的视野。临近这一限度时，应用会压缩对话，早先的轮次转成摘要，使工作得以继续。一条分隔线标出边界，摘要可展开。若有内容不能进入摘要，就放进文件或 adaptive-plan-record（计划记录），文件不在压缩范围内。

搁置一段时间后返回

隔了数天或数周重开一场对话，它会恢复历史、工作目录、知识库与模式。应用会重新核查外部世界的两件事，而非想当然。

- 若工作目录移动或重命名，应用会警告，让用户重新选择该目录，或改为不挂载该目录继续。
- 若文件自上次工作后已变动，智能体在真实工作中第一步是读取当前文件，而非信赖对那些文件的记忆。

当计划的一个阶段完成时，重新对齐以细化剩余部分。详见[在关键时刻想得更清楚](#)。

应用与外挂

Alt Theory 不止一种形态。

应用

桌面应用是完整产品，即对话界面、理解/工作的能力模型、角色与知识库、内置技能、授权、导入与设置。一次安装，无需其他工具，面向尚未使用代理工具的用户。

面向现有代理工具的外挂

若 Alt Theory 的用户已在用某套代理工具（Pi、Claude Code 或其他），外挂形态把 Alt Theory 的方法带入已在用的工具，包含以下内容。

- 内置技能原样不变，因为技能在各套工具间通用
- 一份按所用工具自有格式写的智能体定义，承载 Alt Theory 的身份与原则
- 可选地，一个放在工具可读取位置的知识库

仍属应用独有的是产品界面，即模式开关、授权界面、各选择器与导入，由所用工具以自己的方式提供。详见[外挂与能力差异](#)。

应用的构建基础

应用的代理引擎是 [Pi](#)，一个开源代理工具。三件事直接与此相关。

- 对话以 Pi 兼容格式存储，因此用户的工作始终留在本地，不会被锁进 Alt Theory

- 技能遵循生态约定，所以安装的技能在两边都能用，Alt Theory 的技能也能用在别处
- 来自其他工具的对话可[导入](#)并继续

应用与外挂的选择

- 多数读者用应用。
 - 主要想要方法的工具用户用外挂形态。
 - 两者皆用也是实在的选项。研究对话用应用，已在用的地方用外挂。材料与技能可在两者之间共享。
-

以理解为目标

理解是一个目标，不是一种模式。它通常位于理解模式，但当需要实时检索时会转入工作模式。

工作前的对齐

对于定方向的工作，智能体先与用户对齐，而不是凭猜测行动。adaptive-aligning（自适应对齐）发送成批的问题，每个都带着智能体自己的最佳猜测，让提问角度与解读之间的偏差在据此构建任何东西之前就显明。详见在[关键时刻想得更清楚](#)。

选项而非结论

除非用户或情境要求一个承诺，智能体返回的是不确定性空间，包括确定的内容、可用的选项，以及每个选项背后的取舍。一个分层「为什么」层围绕用户的高层与中层需求解释为什么，并在遇到问题时持续提供选项。(planned)

核实

在理解模式下没有实时查核，因此回忆起的信息标注为未核实。下一步需要核实时，切换到工作模式进行带 DOI 可核引用的学术检索。详见[检索、来源与网页内容](#)。

保留长讨论

在停顿点请求对话摘要（conversation-summary），或保存关键状态到文件。详见内置技能。

在关键时刻想得更清楚

关键时刻是研究项目影响格外集中的节点（研究设计、回应审稿人、重构文章、塑造文献综述的论点、解读结果、框定提案）。

智能体在关键时刻的做法

- 直接点出事实、时间与归属错误，而不是绕着同意。
- 铺开材料与草稿对它们提出的断言之间的差距。
- 提出现实的替代方案，附上支撑，而不是一条笃定的结论。

此项工作的技能

- adaptive-aligning（自适应对齐）：大型或含糊工作前的对齐访谈，成批问题各带智能体的最佳猜测。面板调用：/adaptive-aligning。
- adaptive-plan-record（计划记录）：多步工作的活文档。目标在前，阶段宽松持有，每步揭示的内容，已决定、假定与未定各是什么。它能在对话压缩中存活，并交接给后续对话或协作者。
- precise-edit（精确编辑）：接近定稿文本的克制编辑。语言准确而不重构，保留选择，从不擅自填补占位符。

来源

智能体区分文件所含与它所回忆或推断的内容，并告知一次综合动用了哪些文件。一条需要核实、而智能体并不具备的断言，记为未决，而非用信心填满。

探索替代方案

针对相互竞争的设计，用分叉。针对不该带偏主线的旁支问题，用顺带（BTW）。编辑早先的一条消息，会在不同的切入角度下重跑讨论，原线保持完整。当计划的一个阶段完成时，重新对齐以细化剩余部分。

详见[内置技能与处理回复与控制](#)。

以具体工作为主

工作模式的主场是文件操作、检索与文档生成。

附上一个工作目录，就给智能体一个可行动的地方。它工作时，四处可见以下四件事。

- 每个动作在对话中显示为单独一行，可展开看细节（读取、写入、命令、检索、技能使用）。
- 一个改动过文件的轮次，以一张改动文件卡片收尾，列出文件并统计增删。新写的散文给预览，编辑过的散文给前后对照，代码给差异。
- 边界动作（读取工作目录之外、超出内置检索的联网、安装任何东西）弹出授权请求，说明请求的内容与所授予的权限。
- 修订发生在副本或命名清晰的新文件上，智能体告知写的是哪个文件。

文献检索

- 回忆起的项标注为未核实。
- 学术检索返回可核元数据（标题、年份、DOI、引用数、摘要）。
- 抓取的页面引述并附 URL 引用。若只有摘要可用，则仅返回摘要，并附说明。
- 抓取不绕过付费墙。

详见[检索、来源与网页内容](#)。

文档

文档生成创建新文件。需要生成组件的格式（.docx、.pptx、.xlsx）可能要求一次性安装依赖，由引导设置流程提供。

约定

智能体在写入前先读项目结构，并遵循其可见约定（带日期的输出文件夹、计划各就其位、项目根目录不散落任何东西、不擅自重命名或重组）。

具体工作中浮现的概念问题，可在同一场对话中处理。按需在两种模式间切换。

委派有边界的任务

一场牵头对话可把有边界的任务委派给子代理，后者作为独立会话运行，用户可在右侧栏观察并留言。详见[智能体团队与子代理会话](#)。

应对不确定性

应对实际遇到的几类不确定性的功能。每项在本页一行，背后在系统指南中有相应的完整页面。

- **与智能体失谐。** 澄清重置开启一小段重新对齐的过程。(planned)
- **方向未定。** adaptive-aligning（自适应对齐）在定方向工作前访谈用户，成批问题各带智能体的最佳猜测。详见[在关键时刻想得更清楚](#)。
- **理解不完整。** 默认情况下，智能体返回不确定性空间（确定的内容与可用的选项），而非裁决。一个分层「为什么」层解释为什么，关联用户的需求。(planned) 详见[以理解为目标](#)。
- **未经核实的断言。** 每条断言都标注为现场查得（附引用）、记忆提取（标注未核实）、或推断所得。详见[检索、来源与网页内容](#)。
- **上下文长到放不下。** 压缩将早先的轮次凝聚成摘要，由智能体带着向前走，以一条可见的分隔线标出。详见[对话与历史](#)。
- **回到旧工作。** 返回时，智能体重新读取当前的文件，而非信赖陈旧记忆。若工作目录移动，它会警告并让用户重新选择，或改在不带该目录的情况下继续。详见[继续、转向与稍后返回](#)。
- **换个角度试。** 编辑、再次尝试同一提示、或分叉，都会生成在原版旁以等宽开启的第二个版本。两支以上对比，用工作台 A/B 界面。详见[处理回复与控制](#)。

- **尚未收敛的研究。** 引导模式在 adaptive-aligning 之后、以及在演绎或归纳的理论工作期间，提供下一步。(planned)
 - **回复中断时。** 网络错误时，智能体会明显重试并保留已生成的内容，包括部分答案。重试耗尽时，从断点恢复的动作从中断处接续。
 - **功能的多轮保持。** 技能可保持开启而非一次性调用。激活时按钮变深、之后几轮渐淡、约五轮后脱落，双击锁定。(planned)
-

对话与历史

对话列表

左侧面板列出你的对话，按工作目录分组，最近活动居前。从中可以：

- 跨对话标题搜索
- 打开对话继续
- 从行菜单重命名、删除或复制。复制是从结尾处的分叉：一份独立副本，共享完整历史后各自独立
- 查看每场对话在智能体活跃时的状态：运行中、已完成未读、失败、等待授权

新对话在首次交换后自行命名。手动命名永不被覆盖。

续接与修订

打开对话即完整恢复：历史、模式、工作目录、知识库与模型选择都保留。

- 编辑或修订你早先的一条消息。对话从该点继续，使用你的新措辞，后续轮次离开当前线。首次使用时，应用在执行前提示这一行为。
- 从任意消息分叉，在一场相关对话中推进一个方向，原线保持不变。

各选项保留的内容比较见[处理回复与控制](#)。

压缩

当对话接近模型的上下文限制时，应用会压缩：早先的轮次凝缩为一段摘要由模型携带向前，对话继续。

- 压缩可见。一条分隔线标出边界，摘要可展开。
- 运行时显示状态，可随时停止，对话仍然可用。
- 在停顿点用 `/compact` 主动触发。

摘要有损。关键事实写入 adaptive-plan-record（计划记录）或文件，文件不被压缩。

删除对话

删除对话从列表中隐藏。数据不会立即清除：留在磁盘上作为墓碑记录，可恢复至清理时。如果误删，可在清理前从数据目录中恢复该对话。（在托管研究部署中，标记为私密的对话是例外：七天空闲后会被硬删除。）

导出

- 给读者与同行：markdown 形式的 conversation-summary（对话摘要 / 交接说明），或转录的 HTML 导出。
- 给其他工具续接：对话的 Pi 兼容文件，可被另一工具接起。详见跨工具工作。

故障排查

- 一轮被中断（停止、崩溃、网络）：对话仍可用，已完成的动作记录在转录中。
- 重开后对话看起来异常：先刷新一次。若仍异常，见常见问题，并注明对话的年龄与是否经过导入或压缩。

模型、提供方与访问

AI 模型来自你配置的提供方。Alt Theory 提供环境、方法与界面。

模型标记

输入框旁的模型标记显示本场对话使用的模型，并显示其思考强度（已设置时）。悬停查看提供方与用量详情。点击以切换本场对话的模型，变更从下一轮生效。第一条消息前选定的模型保持生效，对话开始时使用。

配置提供方

配置位于设置中的模型，同时也充当首次启动设置界面。两类访问：

- API 密钥，来自 OpenAI、Anthropic、Xiaomi MiMo 等提供方或任何兼容端点。把密钥粘贴到对应提供方条目。密钥本地存储，保存后不再回显。
- 订阅登录。受支持的提供方（OpenRouter、xAI/Grok 与 OpenAI Codex）通过其自身登录流程连接。Anthropic 走 API 密钥，不走订阅登录。

至少有一个有效且激活的提供方才能开始对话。配置多个，按对话选择。密钥属于设置，不属于聊天消息。

如果以上不熟悉，助手会用日常语言逐步引导提供方配置。

Alt Theory 不会自动读取 Pi 的提供方配置。迁移现有 Pi 设置是一次性的引导复制，详见[共享配置与资产](#)。

当前提供方路径

入口	协议 / 端点
OpenCode Go（OpenAI 兼容）	OpenAI chat completions: https://opencode.ai/zen/go/v1
OpenCode Go（Anthropic 兼容）	Anthropic messages: https://opencode.ai/zen/go
小米 MiMo Token Plan（中国）	OpenAI 兼容： https://token-plan-cn.xiaomimimo.com/v1
小米 MiMo API（中国 / 全球）	OpenAI 兼容；粘贴 MiMo 对应区域的端点
Qwen 3.7 Max（百炼）	OpenAI responses: https://dashscope.aliyuncs.com/compatible-mode/v1
OpenRouter	OpenAI 兼容： https://openrouter.ai/api/v1
OpenAI API	OpenAI responses: https://api.openai.com/v1
Anthropic API	Anthropic messages: https://api.anthropic.com

设置也支持自定义 OpenAI 兼容或 Anthropic 兼容端点。密钥可由 Alt Theory 保存，也可引用你填写的环境变量名；应用不会假定某个固定的提供方变量。

磁盘上的文件

~/ .alt-theory/pi-agent/ 下：

```
models.json    providers.<name> = { baseUrl, api, apiKey, models[] }
auth.json      <provider> = { type: "api_key", key }
settings.json  { defaultProvider, defaultModel, ... }
```

models.json 中的 apiKey 可以是环境变量标记，而非密钥本身。普通修改请使用设置页；这里保留结构，是为了让助手和高级用户无需反向阅读代码也能检查或迁移配置。

用聊天机器人配置模型

设置页是常规途径，应用内的帮手也能带你走完。如果你更想把这件事交给已经在用的聊天机器人——ChatGPT、Kimi、DeepSeek、Gemini，或任何能编辑文件的本地代理——复制下面这段提示词粘贴过去即可。

我在用一个叫 Alt Theory 的应用。它从
~/ .alt-theory/pi-agent/models.json 读取模型配置
(Windows: %USERPROFILE%\ .alt-theory\pi-agent\models.json)。
它不会读取 ~/.pi/ 或其他代理的配置。

文件结构如下：

```
{
  "providers": {
    "<提供方名称>": {
      "baseUrl": "<接口地址>",
      "api": "openai-completions" | "openai-responses" | "anthropic-
messages",
      "apiKey": "<密钥，或环境变量名>",
      "models": [{ "id": "<模型 id>" }]
    }
  }
}
```

请先问我有哪个提供方的访问权限、密钥是什么，然后给出我该保存的完整文件内容，并说明我需要先确认哪些事。不要臆造模型 id：提供方条目建好之后，我会在 Alt Theory 的设置里点「获取模型列表」，由它去问提供方本身。

粘贴之前记住两点：API 密钥等同于密码，只交给你信任的工具；提供方条目建好之后，设置页里的「获取模型列表」比任何聊天机器人对当前可用模型 id 的记忆都更可靠。

每会话模型与思考强度

- 会话可携带自身模型覆盖，每次打开都胜出默认。清除即回到默认。
- 思考强度是对话状态，在输入框或模型菜单中从所选模型支持的等级中选择。
- 模型消失（从你的配置中移除，或被提供方下线）不会破坏重开。应用回退到你的默认并说明，原选择被记住，因此模型回来时，对话会拿回它。

费用与用量

模型使用按你的提供方与你的账号计费。应用不加价。输入框旁的上下文环（[见处理回复与控制](#)）显示上下文用量，其提示包含输入输出 token 数与截至当前的提供方报告费用。费用只在智能体正在回复或工作时累计。

故障排查：已配置但不能工作

1. 提供方是否已保存并激活？草稿条目（已保存但没有有效密钥）会被存储但不可用。
 2. 重新打开对话。配置变更应用于新对话与重开的对话，不在轮次进行中悄然生效。
 3. 仍卡住：[助手](#)可检查你的配置，或见[常见问题](#)。
-

工作目录、文件与路径

工作目录是附在工作模式对话上的文件夹。智能体在此读取你的项目文件并创建新文件。

附上

在工作对话创建时选择一个工作目录，或之后追加。选择时使用系统常规的文件夹对话框。对话列表按工作目录分组，因此随着时间推移，列表会反映你的项目结构。

可以为同一对话追加更多目录，用于跨项目与源资料的工作。附上时，智能体获得该目录的访问权限，并读取该目录的自身约定（给智能体的笔记、项目结构），从而按该项目期望的方式工作。

没有工作目录时，工作对话仍有一个私人临时空间用于它创建的文件，但无法看到你的项目。

智能体在不同位置的权限

- 在附上的目录内：按工作需要读取、创建、编辑文件，无需逐步打扰。所有动作在对话中可见。
- 在附上的目录之外：读取未附上的文件或目录会触发[授权请求](#)，你看到路径后决定。对附上目录外执行写入不可用。工作目录是修改的硬边界。
- 在理解模式下：项目目录不会被附上。理解对话保留一个私人工作空间，智能体可在其中保存你要求的内容，但你的文件只能作为附件进入，超出该边界的读取需经授权。

智能体如何对待你的项目

约定属于[内置技能](#)的一部分，且本身可读：

- 先读。智能体在写入前先读懂项目的组织方式，并遵循既有结构而非自己的默认。
- 原件不被触碰。修订文档意味着写入副本或一个命名清晰的新文件，绝不就地修改原件。
- 产出可查找。创建的内容放在合理位置：带日期的输出文件夹放产出物，plans/ 放计划记录。不在项目根目录散落。
- 不擅自重组。智能体不会自行重命名、移动或重组你的文件。它提出建议，你决定。

文件面板与路径

右侧面板以文件树形式显示工作目录：浏览、打开文件查看，或在 Finder / 资源管理器中显示。文件可从面板加入讨论，面板也是工作完成后查看改动的地方。

当智能体提到文件时，倾向于显示相对于工作目录的路径。文档的转换副本以 `_converted` 后缀出现在原件旁边（[详情](#)）。

故障排查

- 文件夹移动或重命名：重开时不会指向失效路径。应用会提示该目录缺失，让你重新选择或继续不使用。

- 看不到某个文件：检查文件在附上的目录内，对话处于工作模式，且文件是在智能体上次列出目录之后新创建（请智能体再列一次）。
 - 智能体写入的文件找不到：那一轮的改动文件卡片与工具行列出了所有写入的文件与路径。从那里查起。
-

知识库与上下文

知识库是智能体在对话中可查阅的精选参考资料：预先选定并整理的领域资料，与任何单一项目的工作文件区分。

- 工作目录是你的项目：你正在处理的文件，附上后智能体可操作。
- 知识库是参考资料：智能体用以充当讨论事实基底的稳定材料。

一场对话可以同时有两者、其一，或都没有。

启用

输入框旁的知识库选择器为这场对话选择知识库。应用自带领域知识库（环境心理学核心是第一个），你可以添加自己的（自定义）。选择无表示没有知识库可用，工作目录与附件不受影响。可以在对话中途切换知识库，从下一轮开始生效。

智能体如何使用知识库

启用知识库并不意味着每轮都查。智能体知道知识库可用及其覆盖范围，且在讨论需要事实基底时凭判断调阅，与决定读取项目文件的方式相同。

故障排查

- 回答忽略知识库：确认选择器上有选项而非无，并考虑问题是否真正触及知识库的领域。智能体不会强加无关材料。
 - 对话中途切换知识库后感觉陈旧：变更从新的轮次起效，之前回答基于当时启用的知识库。重新提出真正重要的问题。
-

处理回复与对话控制

这些控制的核心用途是对照。大语言模型易于附和、对措辞敏感、且基于采样，单一答案不足以信任：需要看到答案在改变提示、提问角度或模型时是否变化。编辑、同提示重试与分叉都生成一份与原版等宽并列的版本，不干扰原版。

上下文环

输入框旁的环显示本场对话已用上下文窗口的比例。提示分解输入输出 token、缓存使用与提供方报告的费用。当环接近填满时，压缩是让对话继续的方式（[见对话与历史](#)）。

运行中的引导

智能体运行时你发送的文本作为引导消息送达，而非以忙为由拒绝。它会在下一步边界处被看到，与 Pi TUI 中轮次运行时的输入方式一致。

生成可对照的版本

编辑、同提示重试与分叉都会在原版旁以等宽位置开启新版本，原版永不被修改。没有确认对话框：这些操作都不破坏原内容。

- 编辑你早先的一条消息：以新措辞从该点重跑讨论。用于测试同一问题的不同提问角度是否改变答案。
- 重试在原位重跑一次失败或空白的尝试，输入框上方。它不开启侧版本，用于恢复，不是对照。
- 再次试同一提示以兄弟版本重跑同一提示。用于了解同一输入下运行之间的差异。
- 分叉从任意消息开启一场相关对话，携带到该消息为止的全部内容。用于在同一累积上下文中推进不同方向。

要一次比较同一任务的两支以上，用工作台 A/B 对照界面，而不是堆叠分叉。

侧边对话

- 顺带（BTW）开启携带当前上下文的侧边对话，用于不该干扰主线的旁支。
- 助手以全新视角开始，不读取你的讨论，用于应用相关问题（[详情](#)）。
- 牵头对话可把有边界任务委派给子代理，作为独立会话运行（[详情](#)）。

侧边对话共用对话样式，但停在一个窄面板中，它们不占 50/50 对照位置，该位置仅用于分叉、编辑与同提示重试。

把侧边对话加入列表

分叉、顺带、助手或子代理可作为独立首类对话加入对话列表。其来源保留为标签（Branch、From BTW、From subagent 等），而非改名。这取代了早期的「提升」操作。

检索、来源与网页内容

实时查核让智能体对你说的话可以与外部世界对照。

工作模式可用的查核

- 学术检索直接查询学术索引（OpenAlex 与 Crossref），返回可核元数据：标题、年份、DOI、引用数与摘要。无需账号或密钥。文献检索的首选，因为带 DOI 的结果是可以交付给同行的引用。
- 通用网页检索覆盖时事、软件文档与机构页面。亦无密钥。
- page-fetch（页面抓取）把特定页面取回为可读文本，令陈述可附源引用。两档：默认快速档，与用于反爬墙期刊站点的较重浏览器档（首次需要时通过引导设置另行安装）。学术与通用检索由 web-search（网页检索）承担，并受 search-policy（检索策略）约束。

工具箱的「在线查资料」入口调用此功能，直接询问亦然。需要安装的检索工具会在使用前通过授权流程一次性提议。

来源规则

智能体给出的每条事实陈述恰好属于三类之一，保持区分：

- 现场查得：工具在本场对话中返回。附上链接或 DOI。
- 模型记忆：回忆所得但未实时核实。标注为未核实（引用显式注明，例如「[作者, 年份?] - 未核实」），或保留不出。
- 推断所得：从其他事实推理而来。明确说明。

两条推论你会注意到：

- 少数结果是有偏样本。一条片段支持「一个来源说」，不是「领域共识」。当结果冲突时，你会看到分布，而不是悄悄选一个。

- 综合报告会标明它实际读了哪些来源。「读了 40 篇 PDF」就是 40 篇，否则不会这么说。

任何系统都无法杜绝编造。未标注的断言是值得上报的 bug。

付费墙与访问限制

抓取不绕过付费墙。若页面只向公众展示摘要，智能体拿到的就是摘要，并说明「仅摘要」。期刊站点的反爬墙往往可通过保护档抓取，真正的付费墙则不能。

理解模式：无实时查核

理解模式没有实时查核，触及范围受限正是该模式的设计意图。对于可核对的事实，智能体会按可信度标注其当前最佳理解（已确立、合理但未核实、或未知），并标记需要核实的部分，且绝不发明无法核实的引用。当核实成为下一步时，切换到工作模式。理解模式下工具箱的查资料入口显示一行说明，而非被禁用的按钮。

故障排查

- 检索无有用结果：换另一类（学术与通用之间），改写问句，或抓取已知可用的起点页面并顺着其中引用。
 - 抓取失败或返回挑战页面：智能体升级到保护档抓取，这是它的诊断。若保护档未安装，智能体提议安装。
 - 所需工具未安装：引导设置流程处理，经你确认。
-

文档、图像与其他输入

应用读取的内容

文件在工作目录中或附于对话时，可按其真实格式读取：

- Word (.docx)：提取文本。
- PDF：按页面引用提取文本。扫描版或纯图像 PDF 没有可提取的文本。
- Excel (.xlsx)：表格可作为数据读取。
- PowerPoint (.pptx)：按页提取文本。

对于独立或批量转换，智能体可以生成一份文档的 markdown 副本。转换后的文件以 `_converted` 后缀出现在原件旁边，配对关系可见，且原件不被触碰。

附件与目录访问

- 附件将特定文件放到智能体面前，仅供本场对话使用：使用附上按钮，或从文件面板发送。
- 工作目录访问（工作模式）让智能体自己查找并读取项目文件（[详情](#)）。

图像

以同样方式附上图像。智能体能否看到图像取决于模型：支持图像的模型接收图像，仅文本模型看到附上了一张图像并说明无法读取。

应用在模型配置中按模型记录图像能力。如果你知道模型支持图像但应用按纯文本处理，请让助手检查并记录。它会按提供方当前文档验证，而非按模型名称猜测。

生成文档

请求”以文档形式输出”会在工作目录生成一个命名清晰的新文件，从不覆盖你的任何文件。需要生成组件的格式（.docx、.pptx、.xlsx）可能要求一次性安装依赖，通过[引导设置](#)提出，且仅在你确认后才执行。

一条边界：修订你的 Word 文档会输出一份编辑后的副本与一段简明变更说明。不支持输出 Word 的跟踪修订格式。详见[已知限制](#)。

故障排查

- 文档无法读取：检查它能否在其原生应用中打开。受密码保护的文件与损坏的文件无法提取。扫描版 PDF 见上方限制。
- 图像被忽略：模型可能仅文本。检查其记录的能力，或运行图像支持检查并重新对话。
- 转换结果异常：转换优先保留可读文本，弱化版式保真度。对版式敏感的工作，把原件作为真值来源。

技能如何工作

技能是一份可读的指令文件，塑造智能体处理某类任务的方式：何时检索、如何引用、如何编辑接近定稿的文本、如何组织它创建的文件。

技能的所有行为都在技能文件中以日常语言写就。你可打开任一文件，精确阅读智能体被告知内容。你可以审计一个行为背后的方法，加入你自己的技能（作为同等角色参与），且技能在遵循相同约定的工具之间是可迁移的。

技能的来源

三类来源，都在设置中可见：

- 内置：随产品发布，承载其方法（[目录](#)）。
- 用户安装：放在标准技能文件夹中（[方法](#)）。
- 项目技能：在附上的工作目录内。随项目进出（工作模式）。

技能如何被激活

两条路径：

- 匹配。每份技能声明其用途。当你的请求与之匹配，智能体加载该技能并遵循。匹配按语义进行，不按关键词。
- 显式调用。在[命令面板](#)或[工具箱](#)入口按英文 id 调用任何技能（例如 /adaptive-aligning）。目前没有中文别名。

技能加载是可见的：回复中有一行说明当前使用的技能。

技能与模式

只提供指令的技能（如 adaptive-aligning、precise-edit）在两种模式下都可用。方法需要工具的技能（web-search、page-fetch、doc-convert）只工作模式可用，因为理解模式不执行命令。每一份内置技能条目都说明其发挥作用的一侧。

技能重叠

你可以安装与某内置技能同一领域的技能。优先级设置选择「优先内置」「优先用户」或「每次询问」，部分内置技能会在其文本中让位给同类的用户技能。标准详见[添加与管理技能](#)。高级组合见[共享配置与资产](#)。

内置技能

内置技能是 Alt Theory 的方法，写在文件里。每一份都是可读文件，你可以查阅。你自己的技能可以优先（添加与管理技能）。

可调用名是英文 id（与文件夹名、/技能id 一致）。下列每条以英文 id 打头，括号内为中文释义。目前没有中文别名。

查资料

- search-policy（检索策略）规范每次检索：实时查核何时值得使用，三类来源规则（现场查得、记忆、推断），应对稀薄结果，以及无法核实时的回退。两模式都适用，理解模式下它规范无查核时的回退。
- web-search（网页检索，工作模式）做实时通用与学术检索。学术查询返回 DOI、年份与引用数，通用查询用无密钥的网页检索。首次使用可能提议安装一份小型工具。
- page-fetch（页面抓取，工作模式）把 URL 抓取为可读文本以便带来源引用。两档：默认快速档，与用于反爬墙期刊站点的较重浏览器档。不绕过付费墙。

处理文档与文本

- doc-convert（文档转换，工作模式）转换文档（docx、pptx、xlsx、pdf 转 markdown）并生成新的文档文件。转换副本以 _converted 后缀出现，原件不被触碰。
- precise-edit（精确编辑）对接近定稿的文本做克制编辑：语言准确而不重构，保留你的选择，从不擅自填补占位符。
- workspace-conventions（工作区约定）规范智能体创建的文件位置：先读项目结构，带日期的输出文件夹，计划放 plans/，原件不动，不擅自重组。

运行协作

- adaptive-aligning（自适应对齐）是定方向工作前的对齐访谈：成批问题，每个带着智能体自己的最佳猜测，使高置信度的点成为确认。由你主动要求对齐时、或在凭猜测行动代价昂贵时触发。调用示例：/adaptive-aligning。
- adaptive-plan-record（计划记录）是多步工作的活计划，跨越单场对话：目标在前，阶段宽松持有，每步揭示的内容，已定、假定与未定各是什么。存在工作目录中，不受压缩影响。

- `conversation-summary`（对话摘要）把当前对话转成一段保存的 markdown 摘要或交接说明，保留来源，留待问题保持开放。
- `imported-session-context`（导入会话上下文）规范导入的对话：读取当前文件，绝不发明导入未携带的内容，从导入记录中回答「遗失了什么」的问题。

应用自身

- `alt-theory-help`（应用帮助）是唯一面向用户的帮助技能：从当前文档回答，使用运行时状态，对无法核实的部分明确说明。它内部也包含需确认的设置流程和模型图像能力检查；这些不是另外两个技能。

委派

智能体团队工具集（`spawn_agent`、`send_to_agent`、`check_agent`、`wait_for_agents`、`interrupt_agent`、`list_agents`，子代理仅可使用 `message_parent`）是子代理委派的底层机制，不是技能。详见[智能体团队与子代理会话](#)。

添加与管理技能

安装

没有应用市场。技能是包含可读文件的文件夹，把它放进应用查找的位置即可完成安装。

- 给所有工具使用（推荐）：`~/.agents/skills/<skill-id>/`（文件夹名即英文 id，例如 `my-lit-review`）。Alt Theory 与其他遵循同样约定的工具都会找到它。
- 放在项目内：工作目录里的 `.agents/skills/`（或 `.pi/skills/`）文件夹。技能在该文件夹被附上（工作模式）时生效，并随项目移动。

技能文件夹包含一份 `SKILL.md`，说明技能的用途与做法，以及可选的参考文件。这就是全部格式。技能是智能体遵循的指令，所以请安装你读过或信任的技能。

启用

设置中的技能菜单列出所有已发现的技能（内置、用户的、项目的），并控制各技能在何处加载。

- 启用按模式分别设置。理解模式与工作模式各有独立的启用集合。理解模式默认不加载外部技能，工作模式的默认集合更广。
- 修改立即生效。新对话使用新设置，已打开的对话在重开后采用。

优先级

技能优先级是全局设置三档：优先内置（默认）、优先用户、或冲突时询问。冲突指同一类别（两个检索类技能），不是任意两个技能。多个内置技能对同类用户技能让位（doc-convert、page-fetch、web-search）。细粒度规则见[共享配置与资产](#)。

故障排查

- 一个技能始终未触发：在[命令面板](#)中按英文 id 调用它（例如 /my-skill），然后检查它的 SKILL.md 描述。触发匹配的是该描述。目前没有中文别名。
 - 错误的技能胜出：检查优先级设置，检查两者是否属于同一类别。不相关的技能会同时加载。
 - 项目技能缺失：确认该文件夹已附上，且对话处于工作模式。
-

命令

在输入框中输入 / 打开命令面板，按名称可调用的所有项目的完整列表，键入时即时说明。

面板里的名称是英文 id，与技能文件夹名、实现中的命令字符串一致。目前没有中文别名；键入中文名不会匹配。

面板内容

- 所有启用的技能，按 /技能id 调用，例如 /adaptive-aligning、/conversation-summary。完整目录见[内置技能](#)。
- 对话命令：

- `/branch`（分叉）从此处分叉本对话
- `/btw`（顺带）开启携带当前上下文的侧边对话
- `/new`（新对话）开启新对话
- `/compact`（压缩）在你选择的停顿点立即压缩本对话
- `/helper`（助手）在侧边向助手提问

面板在你键入时过滤，每条都说明其作用。

命令与句子

面板里的命令不是执行某一操作的唯一方式。「在这里分叉」「总结这段对话」作为普通句子同样有效。面板的存在是为了精确、速度与发现。用哪种取决于当下。

工具箱

工具箱是输入框旁的小菜单：覆盖超出键入之外最常用操作的一组精选动作。

各条目

- 询问 Alt 如何工作：打开[助手](#)，一个用新上下文回答应用相关问题的助理。
- 在计划或决定上对齐：调用 [adaptive-aligning](#)（[自适应对齐](#)），智能体在定方向工作前访谈你以建立共同理解。
- 计划与记录：调用 [adaptive-plan-record](#)（[计划记录](#)），用于值得跨对话追踪的工作。
- 在线查资料（工作模式）：调用[实时检索](#)（技能侧为 `web-search / page-fetch`）。在理解模式下此项不是一个被禁用的按钮，而是一行说明，理解模式没有实时查核，需切换到工作模式才能搜索。
- 全部技能：跳到[命令面板](#)，且已预输入 `/`。

待发与即发

在工具箱或面板中点击一个技能，会把输入框填充为 `/技能id`（英文 id，例如 `/adaptive-aligning`）并聚焦，以便在执行前输入问题。只有无需提示的命令（`/new`、`/branch`、`/btw`、`/compact`、`/helper`）在点击时立即执行。

工具箱不是什么

它不是技能系统本身，只是精选的入口。每条目都有等效的命令或普通句子版本，未在此列出的内容用一个 / 即可到达。

权限、授权与智能体活动

智能体可做什么、什么始终需要询问、你如何看到正在发生的事。

模型，分层

1. 理解模式：不执行命令、不访问实时网络、不挂载项目文件夹。对话、附件与所选知识库是全部界面。超出此处的读取触发授权。
2. 工作模式，在附上的目录内：执行无需逐轮询问。读取、写入、运行技能描述的命令。
3. 工作模式，在边界处：以下始终需要询问。读取你的文件夹之外的内容，超出内置检索与抓取之外的联网，安装任何东西，策略标记的操作（文件系统破坏、权限提升、凭据路径访问、危险命令如 `rm`、`curl`、`ssh`、`chmod`，逃出工作空间的路径，可疑网络目的地）。

授权请求

对话暂停并展示该操作（正在尝试什么）与范围（允许意味着什么）。选项至少包含「允许一次」与「拒绝」，策略支持时提供更宽范围（本次会话允许，含 30 分钟窗口）。

- 拒绝：智能体另寻路径或说明无法做什么。对话继续。
- 无响应（超时或关闭对话）：视为拒绝。

这是什么

策略层按规则检查操作，授权流程让你参与边界。这是可信代码中的检查，不是操作系统沙箱。两个推论随之而来：

- 检查用于在智能体做真实工作时施加监督，不是用于隔离恶意软件。

- 你安装的扩展与工具以你的用户权限运行。策略约束操作，不能让不可信的扩展变得安全安装。安装你信任的，正如任何程序。

每次拦截与授权都追加到会话级安全审计日志。

查看智能体在做什么

- 在对话中：每个动作以一条工具行实时出现。改动文件的轮次以改动文件卡片收尾。
- 跨对话：列表上每行的状态标记（运行中、已完成未读、失败、等待授权）。当窗口位于后台时，完成、失败或被授权打断的运行会触发系统通知。

故障排查

- 被拒绝后停滞：说明你想让它做什么。附上该文件夹，或在重新请求时授权。
 - 授权了后悔的内容：「允许一次」用完即失效，会话范围会过期。
 - 中途被打断：已完成的步骤在转录中。
-

智能体团队与子代理

牵头对话可以把有边界的任务委派给子代理。子代理是真实的会话，不是后台任务：它们有自己的对话历史、运行谱系与右侧栏可见性，并可以直接发消息。

生成与发消息

牵头方获得一个小的委派工具集。

- `spawn_agent` 启动一个子代理处理有边界的任务
- `send_to_agent` 给它发一条消息
- `check_agent` 与 `wait_for_agents` 读取进度或阻塞等待
- `interrupt_agent` 取消它
- `list_agents` 列出该牵头方下的所有子代理

子代理只有 `message_parent` 一个接口，用于回报。最多三个子代理同时运行，后续启动按顺序排队。

模式与模型

子代理的模式受限于其父对话。理解模式的父对话只能生成理解模式的子对话，工作模式的父对话默认把新子代理设为理解模式。子代理的模型层级可以低于、等于或高于父对话的层级，按你配置的模型与成本比较后确定，无法解析时回退到同一层级。

子代理显示的位置

子代理在右侧栏打开，与顺带或助手侧边对话一致，不在对比面板中（对比面板用于分叉、编辑与同提示重试）。流式输出在键入时按 Markdown 渲染。在侧边对话上方的按钮行内切换不同的子对话。

把子代理加入对话列表

子代理可以作为独立对话加入对话列表。其来源标记保留为「来自子代理」而非改名。这与分叉和侧边对话的「加入对话列表」操作相同。

故障排查

- 子代理似乎卡住：使用 `check_agent`，必要时 `interrupt_agent`。子代理的部分工作保留在它自己的会话中。
 - 子代理已完成但你没看到：结果作为消息留在它的会话中，可从侧栏或对话列表读取。
-

导入、交接与跨工具连续性

从其他工具导入对话并续接，离开时把工作交接出去。

导入

支持来源：Claude Code、Codex、OpenCode、Grok Build（Pi 格式会话直接导入，详见[跨工具工作](#)）。导入入口与创建对话入口在同一位置。

流程：

1. 发现：列出本机已知来源的对话，可检索，按时间倒序。
2. 预览：在创建任何内容之前完整检查所选对话，让你看到导入会携带什么。
3. 导入：转成普通的 Alt Theory 对话：可读历史、可续接、以来源命名。

源不被修改。导入只读，不写回。

导入携带的内容

- 携带：你的消息与助手回复、工具活动、可恢复的图像与附件，源中可读存储的压缩摘要。
- 标注而非伪造：源记录在案但导入无法重放的内容（无法导出的附件、提供方私有状态）以标注占位符出现，明确说明当时是什么。完整源记录作为可检索证据一并保留。
- 拒绝而非编造：导入器无法如实呈现的源会被拒绝，并说明原因。

转换记录与对话一并存储，智能体由此回答「遗失了什么」。

续接导入的对话

第一轮接续线索。智能体把导入的上下文视为可用，因为世界可能已经变化，它在真实工作中的第一步是读取你的当前文件，然后询问看起来已变化的具体事项。重新导入一个已变化的源会创建新副本，永不合并进你的续接。

交接工作出去

- conversation-summary（对话摘要与交接说明）（[内置技能](#)）以 markdown 形式附上来源落地，可由协作者或另一工具阅读。
- 对话以 Pi 格式存储，可在 Pi 中续接并来回。详见[跨工具工作](#)。

核验与故障排查

- 这次导入携带了什么？问智能体，或查看对话的导入记录。
 - 不在导入列表中：源的数据必须在本机，且源必须被支持。详见[常见问题](#)。
 - 导入被拒绝：显示的原因属实，通常是源记录不完整或含糊。源对话保持原样。
-

助手与界面内引导

助手

一个用于 Alt Theory 自身问题的侧边对话：原理与机制、设置所在位置、提供方、密钥与工具的引导式配置。

- 它以新会话开始，不读取当前对话。这让助手可在任何场景复用：空白屏幕上与项目进行中都一样。需要保留当前上下文的旁支问题，用顺带。
- 助手按当前文档回答。具体或可变的内容会查阅产品文档，对无法核实的步骤宁愿承认而不编造。

从工具箱（询问 Alt 如何工作）打开，或用 `/helper`。如果助手对话的内容对你项目有用，将它加入对话列表即成为普通对话。

引导式配置

配置任务（缺少的工具、要配置的提供方、可选的期刊访问浏览器层）仍走同一个助手入口。它会用日常语言说明要安装什么、能做什么、大约多大，由你确认或拒绝，然后验证是否成功。被拒绝的安装可走不安装的替代路径，存在的话。

引导说明在任何地方都能用，助手在任意对话中回答配置问题。执行安装是需在工作模式中进行的动作，从理解模式会指向模式切换。

适时教学

- 首次说明在第一次使用历史重写动作（修订、首次分叉）时出现一次。
- 轮换提示在你已经等待时（智能体思考或长工具运行时）显示单行能力，约 2 秒后开始，只展示已发布的、用户可控的能力。
- 命令面板自带教学：`/` 就是应用的索引。

助手不够时

问题见[常见问题](#)，限制见[已知限制](#)，bug、反馈或研究计划见[README](#)。

设置与配置

设置集合应用级配置。每场对话的选择（模式、模型、角色、知识库、工作目录）存在对话中，不在设置中。设置屏列出自己的若干区块，本页只覆盖打开设置屏幕不直观可见的内容。

修改何时生效

- 新对话立即采用设置变更
- 已打开的对话在重开后采用

谁拥有哪份配置

- 应用拥有自身的模型、提供方与技能启用配置，存于本地目录。它不镜像其他工具的配置。模型配置位于 `~/.alt-theory/pi-agent/`，不是 `~/.pi/agent/`。
- 应用从生态中读取：标准跨工具位置的技能、附上工作目录中的项目资源。是发现，不是同步。
- 工作目录在附上时贡献其上下文与技能。
- 已有 Pi 或其他工具配置可一次性迁移。助手检查你的配置并显式带过有用的部分。优先级：共享配置与资产。

配置在磁盘上的位置

应用所有数据（配置、对话、记录）存于一个本地应用数据目录（数据去向）。模型与提供方配置以 Pi 兼容文件存储，每场对话记录它创建时所用的资产与配置，使历史对话在设置变更后仍可解读。详见[兼容性、更新与集成调试](#)。

常见问题

跨特性的问题与疑问。已确定涉及哪个特性时，优先回到该特性在系统指南中的页面，路径更短。

对话无法启动

问题几乎总在提供方配置链。按模型、提供方与访问中的清单逐项排查，包括提供方已保存并启用、密钥有效且有余额、模型 ID 仍可识别，然后重新打开。没有有效的提供方，应用不会开启对话，修复点在设置中，不在重试。

检索、抓取或文档读取反复失败

- 全部失败，问题出在工具本身。是否已安装？可让智能体检查其检索工具。是否网络屏蔽了目标站点？
- 单个文档失败，见文档、图像与其他输入。

重开后对话看起来异常

尾部缺失、渲染异常、模型不符，先刷新一次，再查看是否有可见提示（工作目录失效、模型回退、资产回退），每条提示说明发生了什么与如何处理（参考）。导入的对话中带占位块，那是标签显示，不是损坏（占位符的含义）。

引用是否会被编造

引用来自带 DOI 的实时学术检索，或被明确标注为未核实。记忆内容不会以检索结果的形式呈现。「无法核实」本身就是正常的回答。详见检索、来源与网页内容。没有任何系统能让编造彻底消失。在这里，编造违反产品的明确规则，不是流畅度的可接受代价。

模型的选用

应用与模型无关，质量跟随模型，因此判断密集的工作应选用预算允许下最强的模型。产品的机制（来源标注、不确定性标注）在各模型间都成立，因为它们由产品自身的指令而非模型层级强制。配置多个并按对话切换。日常用便宜的模型，关键时刻用强模型，是合理的搭配。

费用情况

本软件免费开源。模型按用量由各提供方按其定价收费，应用不加价，并在过程中显示用量与费用。

数据隐私

所有内容都存储在本机。离开本机的内容，仅限于发往所配置模型提供方的请求，以及工作模式下的检索流量。简明数据说明在[安装页](#)。对于在数据使用协议下工作的研究用户，本地存储是默认，托管模式需显式启用并设置相应环境变量。

与通用助手的差异

三点结构性差异：行为面向研究判断而非讨好性回答，理解/工作边界由用户掌控而非一刀切的代理访问，采用本地优先存储配合可读、可移植的格式。详见[Alt Theory](#)简介。

与 Claude Code 或 Pi 的差异

共享同一生态，并可从[它们](#)导入，但定位不同，主打以理解为先的研究工作，代理机制则承载在为学者设计的界面与行为模型中。已在工具生态内深度工作的用户，可能觉得[外挂形态](#)更贴合。

多语言使用

对话可以多语言进行，智能体按用户使用的语言回答。界面随 alpha.6 提供英文、简体中文（zh-Hans）与繁体中文香港（zh-Hant-HK）。文档以英文为主。

配置问题、提供方限制，还是 bug

最省时的分级思路如下。

1. 在新对话中用默认设置复现。如不复现，配置问题，与工作默认对照（[诊断顺序](#)）。
2. 同提示，换不同模型或提供方。如随提供方变化，则是提供方限制（速率、上下文大小、能力），修复在提供方侧或模型选择。
3. 在不同提供方下用默认设置仍复现，可能是真实 bug。报告时附上具体的操作、观察到的现象、预期结果、应用版本、平台，以及该对话是否经过导入或压缩。

排查前先确认问题是边界而非故障（扫描版 PDF、跟踪修订输出、云端文档），然后查看[已知限制](#)。

兼容性、格式与已知限制

参考表。列为限制的条目均为当前边界，列出是为便于规划，不承诺何时改变。

平台

平台	状态
macOS (Apple Silicon)	支持 (打包应用与源码构建)
Windows (x86)	支持 (打包应用与源码构建)
其他组合 (Intel Mac、Windows ARM、Linux)	未测试，不作保证，源码构建可能可用但未验证

界面语言

语言	状态
英文	当前
简体中文 (zh-Hans)	当前 (alpha.6)
繁体中文香港 (zh-Hant-HK)	当前 (alpha.6)

对话语言取决于模型，实际不受限制，智能体按用户使用的语言回复。

文件格式

格式	读取	生成
Markdown / 纯文本	可	可
Word (.docx)	可	可 (作为新文件，需一次性安装组件)
PDF	文本提取 (无 OCR，扫描版或纯图像 PDF 不提取)	否
Excel (.xlsx)	可	可 (作为新文件，需安装组件)
PowerPoint (.pptx)	可 (按页文本)	可 (作为新文件，需安装组件)

格式	读取	生成
图像（PNG、JPEG、GIF、WebP）	需支持图像的模型	对话中以 Mermaid 形式绘制图示

导入来源

来源	状态
Pi	原生格式，可直接续接，无需经导入对话框
Claude Code	支持
Codex	支持（加密的推理与摘要不会被重放，已标注）
OpenCode	支持
Grok Build	支持
其他工具	不支持，从源工具导出为 Markdown 是手动路径

导入仅限本地（源工具的数据必须在本机），对源是只读的。边界细节见[导入](#)。

提供方访问

API 密钥适用于任何已配置的兼容提供方（含 Anthropic）。订阅登录可用于受支持的提供方（目前 OpenRouter、xAI/Grok、OpenAI Codex）。配备兼容 API 的本地模型服务器可作为提供方配置。

已知限制

- 不输出带修订标记的 Word。修改 .docx 时，输出一份编辑后的副本和一段简明变更说明，不是 Word 的修订标记。
- 无 OCR。扫描版或纯图像 PDF 没有可提取的文本。
- 无云端文档集成。Google Docs、Overleaf 等不在范围内，只能通过手动导出与导入文件。
- 理解模式无实时检索。理解模式改为使用标注代替（[原因](#)）。
- 不绕过付费墙。付费来源只能拿到公开摘要。
- 不集成参考管理器。文档中 Zotero 或 EndNote 的字段代码在原文件中保留不变（原文件从不被修改），但应用不读写参考管理器数据。
- 一个窗口焦点对应一场对话。多个工作对话可并行运行并报告状态，但只有一个活动视图，重度多代理编排不是本产品的方向。

- Alpha 期变动。界面与内置资产在版本间可能调整，[策略说明](#)哪些受保护、哪些可能改变。
-

术语

本产品的术语集中定义。交叉引用指向每个术语的完整页面。

理解模式，谨慎对话的能力模式，触及范围受限，不执行命令、不访问实时网络、不挂载项目文件夹。是完整的使用方式，而非工作模式的预览。→ [理解模式与工作模式](#)

工作模式，具备实际触及范围的能力模式，包括工作目录、实时检索与抓取、文档生成、命令，均在可见的授权下进行。→ [理解模式与工作模式](#)

对话，使用 Alt Theory 的基本单位。一条讨论与行动的线索，各自带有历史、模式、模型与材料。持久、可接续、可移植。→ [对话与历史](#)

分叉，从某条消息分出的一场相关对话，携带到该点为止的所有内容，原线保持完整。用于对比回复的控制之一。→ [处理回复与控制](#)

对比面板，分叉或同提示重试后，中央区域按 50/50 比例开启的视图，便于并排阅读两个版本的回复。顺带、助手与工作者保持在窄侧栏。→ [处理回复与控制](#)

顺带 (BTW)，主对话旁的侧边对话，携带当前上下文，用于发散话题。→ [处理回复与控制](#)

助手，应用内置的助理，用于回答关于 Alt Theory 自身的问题。每次以新会话开始（不读取当前对话），依据当前文档回答。→ [助手与界面内引导](#)

工作者，一场由牵头对话委派有边界任务的子对话。它以独立会话运行，采用受限的模式，模型层级可低于、等于或高于牵头方，最多三个并行。(planned) 技能（技能按钮、角色强度）扩展这一层，但尚未实现。→ [智能体团队与子代理会话](#)

工作目录，附在工作对话上的本机文件夹，智能体可在其中读写文件，也是其可修改范围的边界。→ [工作目录、文件与路径](#)

知识库 (KB)，经过精选的参考资料，每场对话可选，供智能体在讨论中援引为事实依据。区别于工作目录。→ [知识库与上下文](#)

角色，一种可选的呈现层，包括语气、节奏、教学风格。改变智能体的呈现方式，从不改变其原则。→ [自定义](#)

灵魂，身份层本身，包含智能体的世界观与原则，任何角色都不能覆盖。与其他一切一样，是一份可读的文件。→ [自定义](#)

技能，一份可读的指令文件，规定智能体处理某类任务的方式。内置技能承载产品的机制，用户自有的技能与之并列。→ [技能如何工作](#)

技能按钮 (planned)，一个跨多轮保持开启的技能，而非一次性调用。激活时按钮变深，之后渐淡，双击锁定。尚未进入界面或代码。→ [以理解为目标](#)

命令，在 / 面板中按名称调用的任何内容，包括技能，以及 /branch、/compact 等对话操作。→ [命令](#)

技能与命令的命名，可调用标识符以英文 id 为准，须与文件夹名、slash 面板中的字符串完全一致（含连字符与大小写）。文档写法为英文在前、中文释义在括号内，例如 adaptive-aligning（自适应对齐）、/branch（分叉）。概念类用语（技能、命令、理解模式）可继续用中文。目前没有中文别名；只写中文名无法在 / 面板中激活。

工具箱，输入框旁的常用操作菜单，是入口，不是完整清单。→ [工具箱](#)

授权，当一项操作越过对话边界（外部读取、网络访问、安装）时显示的请求。拒绝总是安全的，未表态视为不同意。→ [权限](#)

压缩，将长对话早先的轮次可见地凝缩为摘要，以便对话在模型上下文窗口内继续。→ [对话与历史](#)

软删除，删除对话会从列表中隐藏，但数据保留在磁盘上，可在清理前恢复。例外是托管的私密会话，在一段时间无活动后会被硬删除。→ [对话与历史](#)

导入，把另一工具（Pi、Claude Code、Codex、OpenCode、Grok Build）中的对话带入 Alt Theory，作为普通的可续接对话，对损失部分明确标注，而非掩盖。→ [导入与连续性](#)

外挂，Alt Theory 的资产打包后供其他工具使用，内置技能保持原样，外加一份按该工具格式编写的智能体定义，承载产品身份。→ [外挂与能力差异](#)

Pi，应用所基于的开源代理工具。对话、技能与配置遵循生态约定而非自有约定，原因正在于此。→ [应用与外挂](#)

模型 / 提供方，为对话提供能力的 AI 模型，以及按用户账号提供该模型的服务。应用与模型无关，每场对话可选。→ [模型、提供方与访问](#)

来源标注，每条断言均按来源标注，包括现场查得（附引用）、模型记忆（标注为未核实）或推断所得（明确说明）。→ [检索、来源与网页内容](#)

外挂与能力差异

面向在应用与外挂形态之间选择，或同时使用两者的读者。

外挂形态

外挂是 Alt Theory 的资产按所用工具的标准转换而来，并非应用的移植。

- 内置技能保持原样，即与应用发布的是同一份方法文件（目录）。
- 按所用工具自有格式编写的智能体定义，承载 Alt Theory 的身份与原则。
- 可选地，一份放在所用工具可读取位置的知识库。

能力差异

能力	应用	外挂于用户工具
理解优先行为（原则）	内建	通过智能体定义
内置方法技能	内建	同一份技能，安装到工具
理解/工作模式边界	由应用强制	由工具的权限模型控制
授权界面与策略层	应用的	工具的
检索/抓取/转换工具	内置 CLI 工具通过技能调用	工具的原生工具，或同一份 CLI
知识库选择器、角色界面	应用界面	手动
对话列表、分叉/顺带/助手界面	应用界面	工具的
智能体团队与子代理会话	应用功能	工具的委派
来自其他工具的导入	应用功能	不包含

方法技能（search-policy、adaptive-aligning、adaptive-plan-record、precise-edit、workspace-conventions）以纯指令形式迁移。三个工具驱动的技能（web-search、page-fetch、doc-convert）指定具体 CLI 工具，这些必须安装在宿主上，或替换为所用工具的原生工具。search-policy 两种方式都可移植，它指明所需工具的类型。

不能迁移的内容

所用工具的权限模型取代理解/工作模式。如果所用工具默认开放广义工具访问，外挂只带来 Alt Theory 的判断力，不带来应用的强制机制。请将所用工具的权限调整到与之匹配。

选择

- 需要方法，且工作已在工具中，外挂。
- 需要强制的环境、研究界面与 UI，应用。
- 两者都想要，受支持。共享位置存放的技能同时服务两者，对话可在两者间移动。

可用性

外挂包装跟随应用的发布线，各工具的当前安装说明见仓库。(planned)

跨工具工作与连续性

在应用与其他代理工具之间双向迁移工作。

会话主体是 Pi 兼容的

Alt Theory 的对话以 Pi 的 JSONL 会话格式存储，应用没有另造一套会话格式。本机对话可导出给 Pi 续接，兼容的 Pi 会话也可以导入。会话是本机磁盘上的一种文档化格式。不属于对话的应用层内容（显示状态、研究记录、应用元数据）存放在会话主体之外，便于跨工具流转。

按重度用户所需的深度导入

用户级页面介绍流程。背后的机制在边角情况下用得上，主要包括以下几点。

- 三种表征保持区分，即可见的转录（人所见）、活跃的上下文（模型携带的内容）、原始来源证据（保留、可检索、绝不重放）。混同这三者，正是工具虚构历史的根源。

- 生成任何内容之前先做完整预检，即先完整解析并验证整个源，只有通过验证的投影才成为会话。
- 声明式转换，每个有损步骤均被记录，占位符准确说明哪些未被重放。
- 按源类型分别处理。加密的压缩摘要不会被解密，续接从可读的后缀开始，并标注限制。来源谱系不明的源会被拒绝。子代理或子任务的转录归档为证据，不重放。
- 重复导入分类为未变化（重开）或已变化（新副本）。已变化的源不会合并进续接。

在其他工具中续接应用对话

在 Pi 中续接的 Alt Theory 对话保留会话主体，但脱离应用层（理解/工作的强制、授权、应用资产）。智能体身份只有在用户主动带入时才会迁移（外挂）。对人类读者或非代理工具，Markdown 摘要与 HTML 导出是合适的载体（可读，但不是续接格式）。

迁移与保留

内容	迁移？
对话主体（轮次、工具、图像）	是，Pi 兼容 JSONL
压缩摘要	是，源中可读的部分
技能	是，经由共享位置（ 详情 ）
模型/提供方配置	否，各工具各自持有，迁移一次
模式、授权、应用设置	否，应用层
应用级记录（显示状态、来源标注）	随应用副本保留

一个现实的工作流

导入一个 Claude Code 会话，在理解模式下用知识库检视它，在工作目录的计划记录中推进设计，再回到编码工具继续实现。计划记录与文件在项目文件夹中，对两边都可见。跨工具之间天然的共享基础是工作目录，而非对话。

共享配置、技能、模型与资产

当 Alt Theory 与 Pi 或其他工具并列时涉及三个维度，即所有权、发现、优先级，分别对应谁拥有哪份配置、从哪里读取什么、同名项谁胜出。

所有权

- Alt Theory 拥有自己的模型/提供方/认证配置、应用设置（技能启用、优先级、自动命名）、内置资产（技能、灵魂、角色、知识库），以及对话数据，全部存放于自有目录。模型配置位于 ~/.alt-theory/pi-agent/。
- Alt Theory 读取但不拥有共享的跨工具技能位置（~/.agents/skills/），以及所附工作目录内的项目资源。
- Alt Theory 不触碰其他工具的配置目录。它不会将运行时指向 Pi 的配置目录，也不同步。把已有设置带过来是一次性引导式迁移，由助手检查用户的配置并复制有用的部分。

原则是就地读取生态内的资源，持有自己的状态，绝不镜像他人的。

配置实际位置

目录	Alt Theory 读取？	所有者
~/.alt-theory/pi-agent/	是（自有模型/认证/设置）	Alt Theory
~/.agents/skills/	是（共享技能）	共享，跨工具
~/.pi/agent/	否	Pi
附上工作目录内 .agents/skills/、.pi/skills/	是，附上时	项目

发现

来源	加载内容	何时
内置（agent-assets/）	Alt Theory 的技能、灵魂、角色、知识库	始终（受模式/启用约束）
应用级启用	按模式启用的外部技能	受技能设置控制
共享跨工具目录	~/.agents/skills/ 中的技能	启用时
附上工作目录	项目技能与上下文文件	工作模式，附上时

每场对话在创建时都精确记录加载了哪些资产、从何处加载，以便日后回答「它当时为何如此表现」([检视](#))。

迁移已有的 Pi 或其他工具配置

1. 打开助手，让它检查现有配置。
2. 它读取源配置（例如 Pi 的 `~/.pi/agent/`），列出值得带过来的项（包括提供方、模型定义、技能）。
3. 用户确认要复制的内容。
4. 助手把副本写入 Alt Theory 自有的文件，位于 `~/.alt-theory/pi-agent/` 与共享技能目录。源不被修改。

这是一次性迁移，不是持续同步。

优先级

优先级由两层机制叠加处理：

1. 优先级设置（全局，三档），即优先内置、优先用户、或冲突时询问。默认优先内置。冲突指同一类别（如两个检索技能），不是任意两个技能。
2. 技能文本让位，部分内置技能对同类用户技能让位（`doc-convert`、`page-fetch`、`web-search`）。

非技能资产没有冲突，模型配置是 Alt Theory 自有的，角色或灵魂以对话所选为准，项目上下文文件全部加载。

与内置方法的技能组合

- 让横切的方法技能（`search-policy`、`adaptive-aligning`、`adaptive-plan-record`、`workspace-conventions`）各司其职。它们薄而与领域无关。
- 用领域技能承担实质内容。它们与方法层组合，其中 `search-policy` 管理来源标注，用户技能定义其领域中何为「好来源」。
- 若与某个内置方法有分歧，可替换它（优先级设为「优先用户」即可）。

不修改 Alt Theory 的自定义

用户想要的大部分改动不需要动应用。行为由可读资产承载，技能、角色、灵魂、知识库、模型配置。每一层都可扩展。

自己的技能

最大的自定义面，详见[添加与管理技能与共享配置](#)。简版，写一份 SKILL.md，放进共享或项目的技能位置，若与内置方法重叠就设置优先级。一份技能可以承载从学科引用规范到完整分析工作流的任何内容。

角色

角色预设改变智能体的呈现，语气、节奏、苏格拉底式与直接之间的取舍、它前置哪些思考动作。推荐在设置添加你自定义的角色目录，放入自定义角色。避免在默认角色目录修改，因为版本升级可能导致覆盖。

角色改变呈现，不改变原则。身份层（灵魂）高于任何角色。一个「保持鼓励」的角色不会覆盖「事实优先于顺从」。一个苛刻的批评者与一个有耐心的导师都是有效的角色，而不会变成不同的产品。

灵魂

灵魂是身份层本身，世界观与原则。它像角色一样是可选取的资产（也可不选），文件与一切一样在仓库中可读。机械层面支持写替代灵魂，但它从根本上改变产品的性格。对于多数自定义目标，角色或技能是合适的工具，默认灵魂就是产品本身。

知识库

新增一个领域，在应用的知识库资产下创建一个 Markdown 资料文件夹，它就出现在知识库选择器中。有用的知识库是经过精选的，不是倾倒的。智能体凭判断调阅，信噪比决定调阅是否有用。领域元数据可以说明范围与预期用途，便于智能体用得恰当。

自定义指令

每场对话的常设指令（要求的术语、领域要求的格式）可以加入指令资产，并在创建对话时选取，介于身份层与对话之间。

模型与提供方

任何具备兼容 API 的提供方（包括本地模型服务器）都可在配置中添加。本地模型意味着对话内容哪里也不去。模型条目声明能力（图像输入与其他），应用会遵守。

边界在哪

以上都是数据，放入资产目录与配置的文件，升级安全、可分享。应用本身行为的差异（新工具、不同的模式边界、UI 改动）超出自定义，跨入修改 Alt Theory。

修改 Alt Theory 与扩展边界

Alt Theory 开源，软件采用 MIT 协议，原创文档与智能体资产采用 CC BY 4.0。可读、可构建、可改。本页讲述接缝在哪里，哪些改动是设计预期的，哪些意味着需要维护一份 fork。

为改动设计的接缝

按触及范围粗排，

1. 资产，技能、角色、灵魂、知识库、指令（上一页）。数据而非代码，升级安全。
2. 配置，提供方、模型、按模式启用技能、优先级。升级安全。
3. 底下的 Pi 生态。应用嵌入了 Pi 代理运行时，因此作为 Pi 生态资源（包、扩展、工具）存在的能力是新增代理能力的应有路径。在那里构建或采用，而不是再教核心新技能。两条边界成立，包必须使用嵌入式运行时的库（要求不同 Pi 核心的包会被禁用并显示可见的诊断，而非悄悄并存），应用的安全层无论哪个扩展提供了工具，都居间处理工具执行。
4. 应用本身，后端、前端、打包。MIT 协议下可随意改动，但需要承担与上游演进的合并，alpha 期内上游演进很快。

扩展姿态

两个决定塑造了上述边界，

- 不做环境扩展的隐式加载。应用不会自动加载环境中存在的任何扩展，扩展仅应用显式注册的位置加载。这是出于安全与可预测性的选择。一场对话的能力来自其配置所说，不来自机器上恰好装了什么。
- 可信软件边界。扩展与工具以用户权限执行。策略层约束操作，但任何策略都不能让一个不可信的扩展变得安全安装。向运行时加入的内容，就是该用户所信任的内容。审计它。

源码构建的实务提示

- 仓库 README 提供当前的构建与测试命令。[安装页](#)有简版。
- 后端测试套件与前端类型检查是改后的快速检查。在用修改后的构建做真实工作之前跑一遍。
- 架构文档在仓库的 `development/architecture/` 中，面向本读者撰写。

改动之后的去向

资产层是设计为可迁移的，技能尤其在多个工具之间通用。超越个人设置仍有价值的技能、角色或知识库，通过复制文件夹即可分享。对于产品本身的改动，仓库的贡献指南是入口。处于 alpha 期的产品，从描述真实研究使用的问题中受益最多。

兼容性、更新与集成调试

维护页，alpha 期内更新可能改动什么，以及如何诊断一个停止正常工作的集成。

alpha 兼容性策略

在首个非 alpha 版本发布之前，产品对自身保有以下规则，

- 用户数据永远不受更新影响。对话、记录与它们的来源标注始终保留。alpha 没有例外。
- 资产仍可能改变。alpha 构建可能重命名或移除内置资产（角色、灵魂、技能）。若一场对话记录中的资产在当前构建中不存在，则回退到当前默认，并明确显示，重开时出现一条提示，说明记录的是什么、当前以何替代。静默替换视为 bug。
- 界面与配置仍可能调整。发布说明会指出何时调整。

从首个稳定版起，记录中的资产在更新之间保持解析，本节到时将相应修订。

更新

- 打包应用，安装新构建。数据目录与应用分离，完整保留。
- 源码构建，拉取、重新安装依赖、重新构建，并以后端测试作为检查。
- 任何更新后，花一分钟浏览 changelog，重开一场最近对话（若用过的资产被重命名，资产回退提示会出现在这里），如果依赖优先级行为，检查设置。

调试集成

自定配置出现异常时（一个技能停止触发、模型表现与昨天不同、对话加载了未预期的资产），应用为每场对话保留足够记录以回答实际发生了什么，

- 加载了什么。每场对话在创建时记录所组装的精确资产，灵魂、角色、知识库、技能（含来源，内置 / 用户 / 项目）、模型与提供方，含内容哈希。某场对话得到的技能版本，有一个明确的答案。
- 自此的变更。配置变更与恢复时的回退做为事件记录在对话上，因此创建与今天之间的漂移可查而非推断。
- 做了什么。会话语料中的工具行本身就是一份日志，每一轮的读取、命令、技能加载。

可行的诊断顺序，

1. 在新对话中复现。多数「它变了」的报告，是过时未关闭的对话对照新对话的设置。
2. 检查对话的加载资产记录与预期是否一致（对的技能？对的来源？对的模型？）。
3. 检查设置中的优先级与启用（按模式）。
4. 检查应用之外的生态，共享技能文件夹是否变了？提供方是否下线了该模型？附上的工作目录中是否有项目技能在覆盖全局技能？
5. 之后才怀疑构建，把复现带到[常见问题](#)。

回滚

- 资产与技能是文件。把自有的放在版本控制下（`~/agents/skills` 的 git 仓库不花成本），回滚就是 checkout。
- 应用，打包构建可重新安装旧版本，源码构建 checkout 旧标签。数据目录在 alpha 线内向前兼容，回滚应用不会回滚对话。

- 已迁移的配置，迁移副本是普通配置文件。引导流程可重跑，之前的状态就是提供方文件在改动前的内容（试验时保留副本）。